

| ANÁLISE PREDITIVA

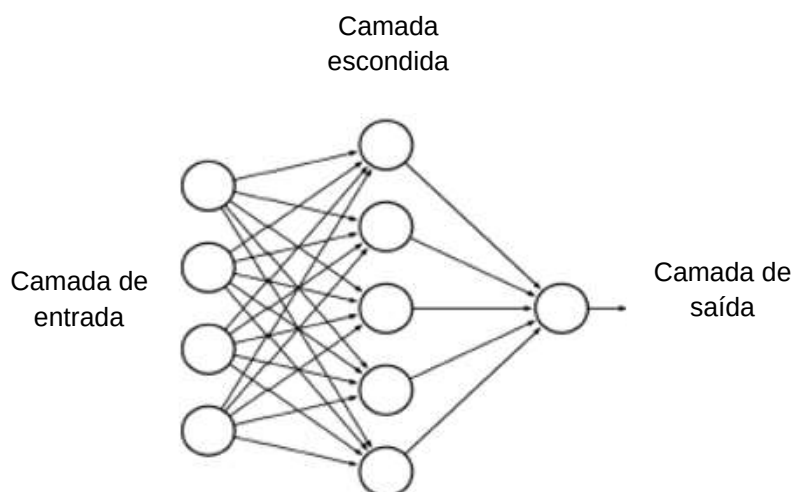
TEMA 1 – REDES NEURAIS PARA SÉRIES TEMPORAIS

O objetivo desta aula é apresentar a aplicação das redes neurais artificiais nas tarefas de predição e estimação, utilizando as diferentes funções disponíveis no Sistema R.

As redes neurais artificiais são ferramentas importantes atualmente, devido à capacidade de “aprender” padrões, por meio de treinamento, tornando, assim, o seu uso fundamental no desenvolvimento da Inteligência Artificial.

Uma rede neural artificial pode ser dividida em três partes, como mostra a Figura 1.

Figura 1 – Rede neural artificial



Fonte: elaborado com base em Ciaburro; Venkateswaran, 2017.

Segundo Silva, Spatti e Flauzino (2010), as três camadas de uma rede artificial são:

- Camada de entrada: responsável pelo recebimento de dados ou medições, provenientes do meio externo. Essas entradas (amostras ou padrões) são geralmente normalizadas (intervalo entre 0 e 1).
- Camadas escondidas, intermediárias ou ocultas: formadas de neurônios (unidades processadores) que têm a responsabilidade de extrair as características relacionados ao processo ou sistema a ser inferido.
- Camada de saída: é também constituída de neurônios, sendo responsável pela geração e apresentação dos resultados finais.

Existem dois tipos principais de processos de aprendizagem: aprendizagem supervisionada e não supervisionada. Na aprendizagem supervisionada, fornecemos as entradas e saídas desejadas. Os pesos são modificados para reduzir a diferença entre as saídas reais e desejadas. Na aprendizagem não supervisionada, apenas fornecemos as entradas. A rede neural ajusta seus próprios pesos, de modo que entradas semelhantes produzam saídas semelhantes. A rede identifica os padrões e diferenças nas entradas sem qualquer assistência externa (Ciaburro; Venkateswaran, 2017).

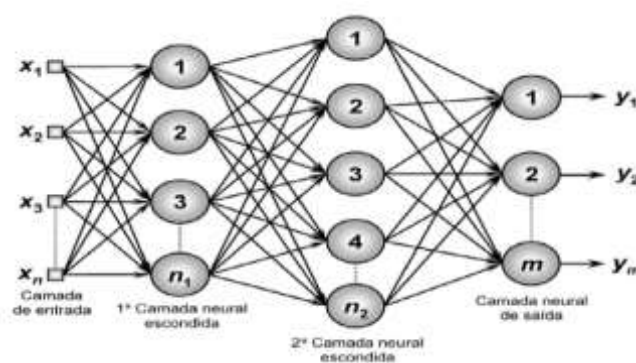
Para saber mais sobre redes neurais artificiais, sugerimos consultar Silva, Spatti e Flauzino (2010).

1.1 Redes neurais para séries temporais

As redes neurais artificiais têm sido amplamente utilizadas para fazer previsões de séries temporais.

Para a previsão de séries temporais é utilizada a rede neural *multilayer perceptron*, apresentada na Figura 2.

Figura 2 – Rede perceptron de camadas múltiplas



Fonte: Silva; Spatti; Flauzino, 2010.

O conjunto total de observações disponível é dividido em dois subconjuntos, um subconjunto de treinamento e outro de teste. O primeiro subconjunto, de treinamento, é composto com cerca de 60% a 90% das observações do conjunto total, e será usado no processo de aprendizado da rede. O subconjunto de teste, com composição entre 40% e 10% do conjunto total de observações, será utilizado para validação da rede (Silva; Spatti; Flauzino, 2010).

A função *mlp()* da biblioteca *nnfor* do Sistema R cria e treina a rede neural *multilayer perceptron*, do tipo *feedforward*. Após a construção do modelo preditivo, deve-se fazer a avaliação do desempenho, por meio, por exemplo, do erro quadrático médio (*mean square error* – MSE), que é a média das diferenças entre os valores reais e previstos ao quadrado. Quanto menor o valor da medida, melhor é o modelo preditivo.

TEMA 2 – REDES NEURAIS PARA MODELOS DE REGRESSÃO

O modelo de regressão é uma técnica para estudar a relação entre uma variável dependente e uma ou mais variáveis independentes.

Um modelo de rede neural é muito semelhante a um modelo de regressão não linear, sendo que o primeiro pode lidar com uma quantidade muito grande de parâmetros do modelo. Por isso, diz-se que os modelos de redes neurais têm a capacidade de aproximar qualquer função contínua.

A rede neural é muito boa para aprender a função não linear e podem ser aprendidas várias saídas ao mesmo tempo. A função *neuralnet()* da biblioteca *neuralnet* possibilita trabalhar a rede neural *multilayer perceptron* na visão da análise de regressão, isto é, para aproximar as relações funcionais entre variáveis independentes e respostas (dependentes).

Para utilizar essa função, é necessário normalizar as variáveis, isto é, colocar as variáveis no intervalo de 0 e 1.

TEMA 3 – MÉTODO DE CLASSIFICAÇÃO K-NN

A técnica de classificação é uma tarefa da análise preditiva na mineração de dados. O algoritmo k-vizinhos mais próximos ou k-NN (*k-nearest neighbors*) classifica um elemento cujo grupo é desconhecido, fazendo a comparação desse elemento com aqueles que têm um grupo já conhecido. O princípio desse algoritmo é armazenar o conjunto de treinamento e realizar comparações entre os elementos armazenados a cada vez que um elemento de teste é apresentado.

Apesar do processo de classificação ser custoso em alguns casos, a lógica que o implementa é simples. Um elemento é classificado pela “votação da maioria”, de acordo com Silva, Peres e Boscarioli (2016), isto é, o grupo da maioria dos k elementos mais próximos ao elemento teste será atribuído a ele.

Normalmente, são usadas métricas de distância para medir similaridade e dissimilaridade entre dois elementos. Porém, antes de calcular as distâncias, recomenda-se fazer a normalização de cada atributo (variável), isto é, colocar no intervalo de 0 a 1.

Existem diferentes formas de medir distâncias entre dois elementos com atributos descritivos do tipo numérico, por exemplo: euclidiana, *Manhattan*, *Minkowski* etc. Será apresentada apenas a distância euclidiana, que certamente é a mais conhecida e utilizada para calcular a distância entre dois objetos no espaço p-dimensional. As distâncias de *Manhattan* e *Minkowski* serão abordadas nos próximos conteúdos.

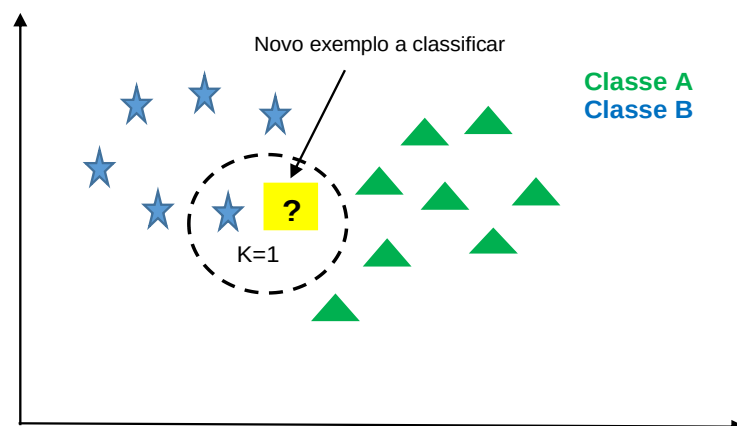
A distância euclidiana entre dois objetos x e y, em um espaço p dimensional, é definida por:

$$d(x, y) = \sqrt{\sum_{j=1}^p (x_j - y_j)^2} \quad (1)$$

Para a execução do algoritmo k-NN, é necessário definir dois parâmetros: o valor de k, que define o número de vizinhos mais próximos consultados no processo de classificação e a medida de distância. Além desses dois parâmetros, é necessário viabilizar um conjunto de treinamento e um conjunto de teste. A escolha do parâmetro k deve ser feita com cuidado (Silva; Peres; Boscarioli, 2016).

A Figura 3 apresenta um exemplo ilustrativo para classificação k-NN. Considerando o número de vizinho (k) igual a 1, deseja-se saber qual será a classificação do novo exemplo representado por ponto de interrogação. Neste caso, deve-se identificar a classificação do vizinho mais próximo, ou seja, estrela, que será a classificação do novo elemento (Rocha, 2019).

Figura 3 – Exemplo ilustrativo para classificação k-NN



Fonte: elaborado com base em Rocha, 2019.

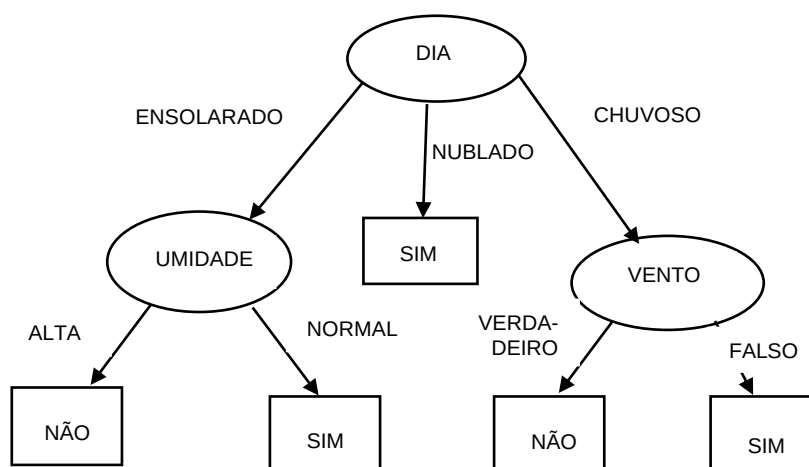
Nota: número de vizinho $k=1$.

TEMA 4 – ÁRVORE DE DECISÃO

A árvore de decisão (*decision tree*) é uma das técnicas mais populares de mineração de dados. É normalmente utilizada para resolver tarefas de classificação de dados. Essa técnica consiste em um conjunto de nós internos e folhas, organizados em um modelo hierárquico.

A árvore de decisão apresentada na Figura 4 ilustra se haverá partida de tênis (sim ou não), que é a saída (Y), de acordo com as variáveis preditivas dia (ensolarado, nublado ou chuvoso), temperatura (elevada, amena ou baixa), umidade (alta ou normal) e vento (falso ou verdadeiro) (Quillici-Gonzalez; Zampirolli, 2014).

Figura 4 – Árvore de decisão



Fonte: elaborado com base em Quillici-Gonzalez; Zampirolli, 2014.

Identificando os nós e ramos na Figura 1, tem-se: nó raiz dia e os ramos ensolarado, nublado e chuvoso; nó interno umidade e ramos alta e normal e nó interno vento e ramos verdadeiro e falso; nó folha: não e sim.

Para a construção da árvore de decisão são calculadas duas medidas, a entropia e o ganho. A entropia é a medida que permite caracterizar o grau de desordem. Quanto menor a entropia, mais organizado é o conjunto de dados. O ganho de informação é a medida da redução na entropia, ocasionada pela divisão dos elementos (exemplos) em relação a um atributo escolhido. Em cada iteração do algoritmo, é escolhido o atributo que apresenta um maior ganho.

TEMA 5 – EXEMPLOS DE APLICAÇÃO

Nesta seção, iremos aplicar as técnicas apresentadas, utilizando as diferentes bibliotecas e funções disponíveis no Sistema R. É importante observar que, de acordo com os objetivos da análise que pretende realizar, deve-se aplicar as técnicas adequadas, bem como observar os tipos de dados necessários, para o sucesso da análise.

5.1 Redes neurais para séries temporais

Considerando a série do índice da produção da indústria de transformação (média 2012 = 100), no Brasil, período de janeiro de 2002 a fevereiro de 2020, o objetivo é realizar um estudo aplicando as técnicas de redes neurais aos dados.

Tabela 1 – Índice da produção da indústria de transformação, Brasil – jan. 2002 – dez. 2011

Meses	Anos									
	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011
Jan.	73,0	74,3	77,3	81,5	83,6	86,9	94,8	78,7	91,0	92,8
Fev.	70,4	72,3	74,5	77,0	80,5	82,7	91,2	76,3	89,1	95,7
Mar.	78,0	77,8	87,9	88,8	92,8	96,8	98,0	88,8	105,6	104,8
Abr.	80,4	77,0	83,0	87,5	85,8	90,8	99,7	85,5	99,6	97,6
Mai	81,2	80,1	87,3	91,5	95,7	100,3	102,6	91,6	104,6	107,4
Jun.	78,1	76,9	87,0	91,9	91,5	97,2	103,6	92,4	102,8	102,9
Jul.	84,1	82,0	91,0	90,9	94,0	100,1	108,8	98,0	107,1	106,2
Ago.	84,4	82,2	93,0	96,3	99,1	105,5	107,1	100,0	108,4	111,2
Set.	83,1	86,6	93,2	92,6	93,6	98,8	107,8	100,0	106,0	105,0
Out.	90,3	91,1	94,5	94,2	97,9	108,7	108,9	106,4	108,2	106,5
Nov.	84,9	85,5	92,9	93,1	96,3	103,1	96,7	102,2	107,3	104,3
Dez.	75,3	78,3	85,3	86,9	86,8	92,3	79,2	94,2	96,2	95,1

Fonte: IBGE (PIM-PF)

Nota: média 2012 = 100.

Tabela 2 – Índice da produção da indústria de transformação, Brasil – jan. 2012 – fev. 2020

Meses	Anos								
	2012	2013	2014	2015	2016	2017	2018	2019	2020
Jan.	88,2	94,5	92,2	85,7	74,6	74,9	79,8	77,8	79,0
Fev.	89,4	88,4	93,0	81,9	74,4	73,7	75,9	79,0	78,6
Mar.	99,8	98,7	97,3	92,7	82,8	83,7	85,0	80,7	
Abr.	92,3	103,2	96,0	86,5	82,1	77,2	85,4	84,6	
Mai	102,2	106,2	101,6	90,9	84,8	88,4	81,9	91,5	
Jun.	98,0	102,3	94,1	90,3	86,6	86,7	89,6	85,6	
Jul.	104,6	109,0	104,1	93,5	88,2	90,6	94,6	93,1	
Ago.	112,5	113,3	105,8	95,4	91,9	95,7	97,5	95,4	
Set.	104,2	108,6	105,3	92,3	89,4	91,8	89,5	91,0	
Out.	112,3	113,7	108,9	95,5	88,7	94,0	94,4	96,8	
Nov.	105,8	106,9	99,1	86,6	84,8	89,4	87,9	87,4	
Dez.	90,8	88,9	84,8	74,6	73,7	78,4	74,3	74,9	

Fonte: IBGE (PIM-PF)

Nota: média 2012 = 100.

Recomenda-se criar o arquivo de dados no Excel e fazer a leitura no R.

```
# Leitura do arquivo em excel
install.packages("readxl") # instala a biblioteca readxl
library(readxl)
# Identificação do diretório e arquivo #
path<-"D:/DADOS/pim-pfind_1.xlsx "
pim<-read_excel(path); str(pim)

attach(pim)
```



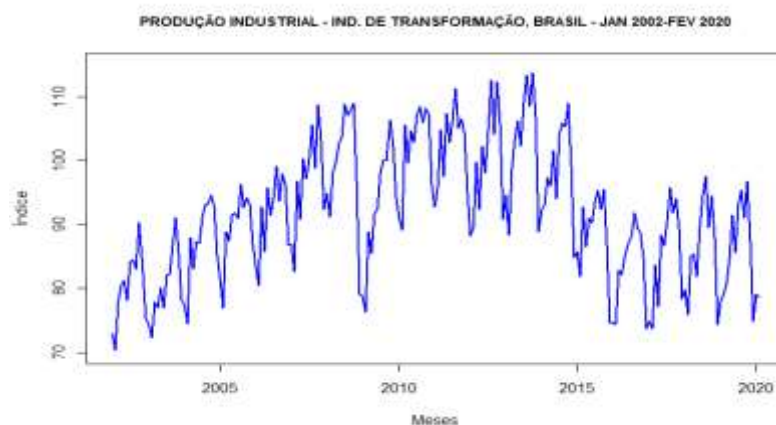
```
# Transformando em classe times series e construindo o gráfico da
# série temporal
library(forecast)

pim<-ts (VALOR,start=2002,frequency = 12)

plot(pim,main='PRODUÇÃO INDUSTRIAL - IND. DE TRANSFORMAÇÃO,
      BRASIL - JAN 2002-FEV 2020',cex.main=0.9,col='blue',
      ylim=c(70,115),lwd=2,ylab="Índice", xlab="Meses")
```

O Gráfico 1 mostra o índice da produção da indústria de transformação, no período de janeiro de 2002 a fevereiro de 2020.

Gráfico 1 – Índice da produção da indústria de transformação, Brasil – jan. 2002 – fev. 2020



Fonte: elaborado a partir dos dados do IBGE, 2020.

Para criar a rede neural, utiliza-se a função *mlp*, da biblioteca *nnfor*. As 196 observações iniciais (jan. 2002 a abr. 2018) foram utilizadas para treinamento e as outras 22 observações (maio 2018 a fev. 2020) para avaliação da rede.

```
install.packages("nnfor") # instala a biblioteca nnfor
library(nnfor)
set.seed(123)
mlp.fit <- mlp(ts(pim[c(1:196)]),start=2002,frequency = 12),
             hd=NULL, hd.auto.type=TRUE,lags=12, reps=20,
             difforder=c(1,12),sel.lag=TRUE)
```

hd: Número de nós (neurônios) ocultos. Pode ser um vetor, em que cada número representa o número de nós ocultos de uma camada oculta diferente;

`hd.auto.type`: utilizado somente quando `hd=NULL`;
`lags`: defasagens da série para utilizar como entrada;
`reps`: número de redes para treinar, o resultado é a previsão do conjunto
`difforder`: vetor incluindo as diferenças de diferenciação. Por exemplo, `c(1,12)` aplicará as diferenças primeiro e sazonal (12). Para sem diferenciação, use 0.
Para seleção automática, use `NULL`;
`sel.lag`: seleciona automaticamente as defasagens. Pode ser *TRUE* ou *FALSE*.

Para fazer a previsão utilizando as 22 observações finais:

```
mlp.frc <- forecast(mlp.fit,h=22)
```

Para visualizar os valores observados e previstos:

```
yprev<-mlp.frc$mean
yobs<-ts(pim[c(197:218)],start=c(2018,05),frequency=12)
y<-cbind(yobs,yprev);y
```

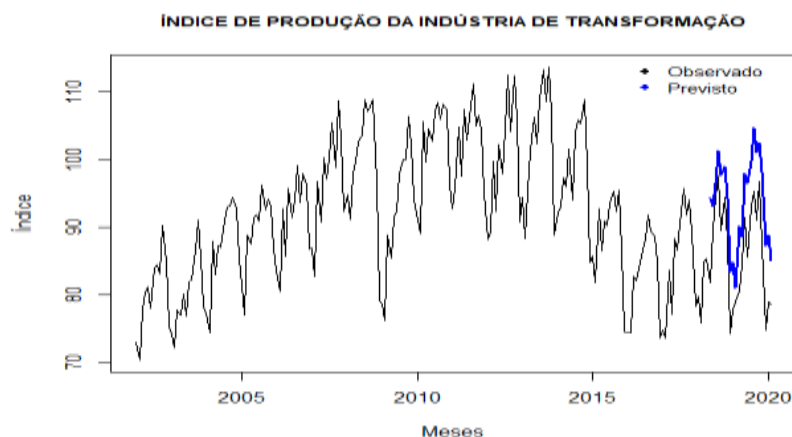
	yobs	yprev
May 2018	81.9	94.26903
Jun 2018	89.6	93.12514
Jul 2018	94.6	96.57085
Jan 2019	77.8	84.86436
...	...	
Jan 2020	79.0	88.85132
Feb 2020	78.6	85.06937

Cálculo do erro quadrático médio (MSE):

```
yobs<-ts(pim[c(197:218)],start=c(2018,05),frequency=12)
MSE<-sum((yobs - mlp.frc$mean)^2)/length(yobs);MSE
> MSE
[1] 63.14315
```

Extraindo a raiz quadrada, tem-se $RMSE = 7,9714$. Os valores observados e previstos são mostrados no Gráfico 2.

Gráfico 2 – Valores observados e previstos do índice da produção da indústria de transformação



Fonte: elaborado a partir dos dados do IBGE, 2020.

Construção do gráfico no R:

```
plot(pim,main="ÍNDICE DE PRODUÇÃO DA INDÚSTRIA DE TRANSFORMAÇÃO",
     cex.main=0.9,xlab="Meses", ylab="Índice")
lines(mlp.frc$mean,col="blue",lwd=2)
legend('topright',legend=c("Observado","Previsto"),pch=18,
      col=c("black","blue"), bty='n', cex=.95)
```

5.2 Redes neurais para modelos de regressão

Será utilizado o arquivo de dados *mtcars* (testes de estrada para automóveis – *Motor trend car road tests*) disponível na biblioteca *datasets*, no R. As variáveis utilizadas são o consumo de combustível (milhas por galão) e deslocamento do motor (em *cubic inches*), que se encontram na Tabela 3.

```
install.packages("datasets") # instala a biblioteca datasets
library(datasets)           # ativa a biblioteca datasets
carros<-mtcars
desloc<-carros$disp
consumo<-carros$mpg
dados<-data.frame(cbind(desloc,consumo)); dados
```

Tabela 3 – Consumo de combustível e deslocamento do motor

Obs.	Deslocamento	Consumo	Obs.	Deslocamento	Consumo
1	160,0	21,0	17	440,0	14,7
2	160,0	21,0	18	78,7	32,4
3	108,0	22,8	19	75,7	30,4
4	258,0	21,4	20	71,1	33,9
5	360,0	18,7	21	120,1	21,5
6	225,0	18,1	22	318,0	15,5
7	360,0	14,3	23	304,0	15,2
8	146,7	24,4	24	350,0	13,3
9	140,8	22,8	25	400,0	19,2
10	167,6	19,2	26	79,0	27,3
11	167,6	17,8	27	120,3	26,0
12	275,8	16,4	28	95,1	30,4
13	275,8	17,3	29	351,0	15,8
14	275,8	15,2	30	145,0	19,7
15	472,0	10,4	31	301,0	15,0
16	460,0	10,4	32	121,0	21,4

Fonte: dados disponíveis na biblioteca *datasets*, Sistema R, 2020.

A função utilizada é a *neuralnet*. Para utilizar essa função, é necessário normalizar as variáveis, isto é, colocar as variáveis no intervalo entre 0 e 1.

A função a seguir fará a normalização das variáveis do arquivo dados.

```
normalize <- function(x) {
  return ((x - min(x)) / (max(x) - min(x)))
}
normaliza <- as.data.frame(lapply(dados, normalize))
```

Após a normalização das variáveis, o arquivo será dividido em duas partes, uma chamada de treinamento e outra de teste. A parte que será utilizada para treinamento consistirá das primeiras 20 observações e, para teste, serão consideradas as observações 21 até 32.

```
treino <- normaliza[1:20,]
teste <- normaliza[21:32,]
```

Em seguida, serão separadas as variáveis consumo e deslocamento.

```
testeX <- teste[,1]      # variável deslocamento
testeY <- teste[,2]      # variável consumo
```

Aplicando a rede neural, tem-se:

```
install.packages("neuralnet") # instala a biblioteca neuralnet
library(neuralnet)
set.seed(123)
```

```
nn <- neuralnet(consumo ~ desloc, data=treino, hidden=c(3,3),
  threshold=0.01,linear.output=FALSE)
```

hidden: um vetor de números inteiros especificando o número de neurônios ocultos (vértices) em cada camada;

threshold: um valor numérico que especifica o limite para as derivadas parciais da função de erro como critério de parada;

linear.output: saída linear, verdadeiro ou falso.

Para avaliar o modelo, será utilizada a parte chamada de teste.

```
nn.results <- compute(nn, data.frame(testeX))
```

Os valores estimados são obtidos por meio de:

```
nn.results$net.result
> nn.results$net.result
      [,1]
[1,] 0.6526644
[2,] 0.2074033
... ..
[11,] 0.2139501
[12,] 0.6481324
```

Esses valores foram obtidos por meio da utilização das variáveis normalizadas. Para retornar aos valores originais, é necessário fazer o processo inverso, ou seja: $x = x_{normalizada} \times (x_{máx} - x_{min}) + x_{min}$

```
prnn<-nn.results$net.result*(max(dados$consumo) -
min(dados$consumo))+min(dados$consumo)
prnn
      [,1]
[1,] 25.73761
[2,] 15.27398
... ..
[11,] 15.42783
[12,] 25.63111
```

Para avaliar o ajuste do modelo, calcula-se o erro quadrático médio.

```
Y<-dados$consumo[21:32]# variável consumo original
MSE.nn <- sum((Y - prnn)^2)/nrow(teste); MSE.nn
```

```
[1] 6.480745 (rede neural)
```

Os valores observados e ajustados:

```
y<-cbind(Y,prnn)
```

```
y
```

	consumo	prnn
1	21.5	25.73761
2	15.5	15.27398
3	15.2	15.39778
4	13.3	15.06899
5	19.2	14.88207
6	27.3	29.71609
7	26.0	25.71398
8	30.4	28.41320
9	15.8	15.06393
10	19.7	22.77469
11	15.0	15.42783
12	21.4	25.63111

5.3 Método de classificação k-nn

Será utilizado o arquivo *computers* disponível na biblioteca *Ecdat*, do *software* R. Esse arquivo contém várias variáveis (características sobre os computadores) com 6.259 observações.

Para a classificação das observações (computadores) de acordo com o tipo do fabricante, se *premium* ou não, serão utilizadas as seguintes variáveis:

price: preço do computador em U\$ dólar

speed: velocidade do relógio (MHZ)

hd: tamanho do disco rígido em MB

ram: tamanho da ram em MB

screen: tamanho da tela em polegadas

```
install.packages("Ecdat") # instala a biblioteca Ecdat
```

```
library(Ecdat) # ativa a biblioteca Ecdat
```

```
dados1<-Computers
```

```
attach(dados1)
```

```
dados<-data.frame(price, speed, hd, ram, screen, premium)
```

```
str(dados)
```

```
'data.frame': 6259 obs. of 6 variables:
```

```
$ price : num 1499 1795 1595 1849 3295 ...
```

```
$ speed : num 25 33 25 25 33 66 25 50 50 50 ...
```

```
$ hd : num 80 85 170 170 340 340 170 85 210 210 ...
```

```
$ ram : num 4 2 4 8 16 16 4 2 8 4 ...
```

```
$ screen : num  14 14 15 14 14 14 14 14 14 15 ...
$ premium: Factor w/ 2 levels "no","yes": 2 2 2 1 2 2 2 2 2 2
```

As variáveis apresentam diferentes unidades, portanto, deve-se inicialmente normalizá-las.

```
normalize <- function(x) {
  return ((x - min(x)) / (max(x) - min(x)))
}
normaliza <- as.data.frame(lapply(dados[,1:5], normalize))
dadosn<-data.frame(normaliza,dados$premium)
```

Em seguida, o arquivo de dados será separado em treinamento e teste, utilizando uma razão igual a 0,75. O subarquivo treinamento será composto por 75% das observações e o restante dos 25%, irá compor o teste.

```
install.packages("caTools") # instala a biblioteca caTools
library(caTools)           # ativa a biblioteca caTools
set.seed(1)
divisao <- sample.split(dadosn$dados.premium, SplitRatio = 0.75)
treinamento <- subset(dadosn, divisao==TRUE) #treinamento
teste <- subset(dadosn, divisao==FALSE) # teste
```

Para aplicar o algoritmo k-NN para fazer a classificação, é necessário definir o subarquivo treinamento (*train*), o subarquivo teste (*test*), o grupo (*cl*) e o número de vizinhos mais próximos (*k*).

É possível fazer simulação para obter o melhor valor para *k*, por meio de:

```
install.packages("class") # instala a biblioteca class
library(class)           # ativa a biblioteca class
previsto = NULL
perc.erro = NULL
for(i in 1:10){
  set.seed(1)
  previsto = knn(train = treinamento[,1:5], test= teste[,1:5],
    cl= treinamento[,6],k=i)
  perc.erro[i] =mean(teste[,6] != previsto)}
```

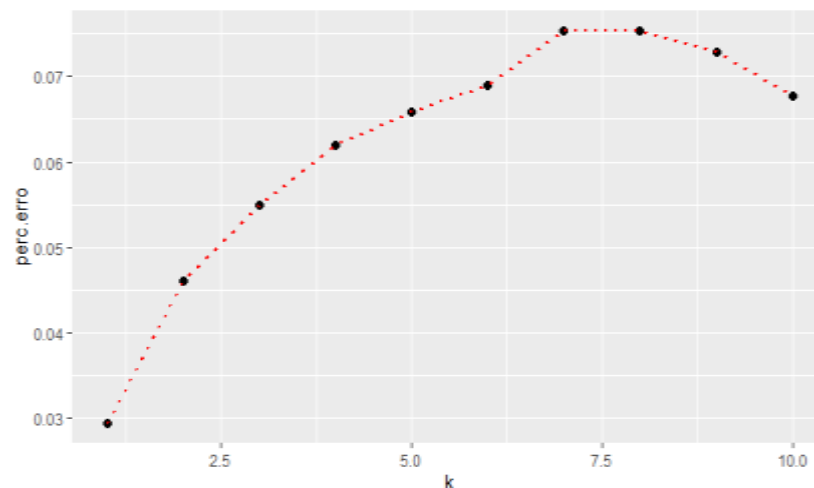
Plotando no gráfico os valores de *k* e percentual de erro:

```
install.packages("ggplot2") # instala a biblioteca ggplot2
```

```
library(ggplot2)      # ativa a biblioteca ggplot2
k <- 1:10
perc.erro <- data.frame(perc.erro,k)
ggplot(perc.erro,aes(x=k,y=perc.erro))+
  geom_point(size=2)+geom_line(lty="dotted",
    color='red', size=0.9)
```

O Gráfico 3 mostra que o melhor valor para k é 1, apresentando menor erro. Portanto, a classificação será feita utilizando k=1.

Gráfico 3 – Percentual de erro em função do número de vizinhos



Fonte: elaborado a partir dos dados da biblioteca *Ecdat*, do sistema R, 2020.

```
set.seed(1)
previsto <- knn(train = treinamento[,1:5], test= teste[,1:5],
  cl= treinamento[,6],k=1)
```

Para verificar a qualidade do algoritmo, é possível comparar a classificação observada e a prevista.

```
install.packages("caret") # instala a biblioteca caret
library(caret)             # ativa a biblioteca caret
tabela<-table(teste[,6],previsto); tabela
```


	no	yes
no	135	18
yes	28	1384

Sendo assim, a taxa de acerto igual a 0,9706.

Para classificar uma nova observação, basta substituir o arquivo teste pelo arquivo que contém os dados da observação que deseja classificar. Por exemplo: $x = (1795, 33, 85, 2, 15)$, correspondendo aos valores das variáveis *price*, *speed*, *hd*, *ram* e *screen*. Obviamente, esses valores devem ser normalizados e armazenados em *xn*.

```
x<-c(1795,33,85,2,15)
xn<-c(0.1566957,0.08,0.002380952,0.00,0.05882353)
classifica <- knn(train = treinamento[,1:5], test= xn,
cl= treinamento[,6],k=1); classifica
[1] yes
```

5.4 Árvore de decisão

Será utilizado o mesmo arquivo do exemplo de aplicação do método de classificação k-NN. O arquivo *computers* disponível na biblioteca *Ecdat*, do *software* R, que contém 6.259 observações e diferentes variáveis (características sobre os computadores).

```
install.packages("Ecdat") # instala a biblioteca Ecdat
library(Ecdat) # ativa a biblioteca Ecdat
dados<-Computers
```

Para a construção da árvore de decisão serão utilizadas as variáveis: *price*, *speed*, *hd*, *ram* e *premium*. Assim, um novo arquivo será criado somente com essas variáveis.

```
attach(dados)
computadores<-data.frame(price,speed,hd,ram,premium)
str(computadores)
'data.frame': 6259 obs. of 5 variables:
 $ price : num 1499 1795 1595 1849 3295 ...
 $ speed : num 25 33 25 25 33 66 25 50 50 50 ...
 $ hd : num 80 85 170 170 340 340 170 85 210 210 ...
 $ ram : num 4 2 4 8 16 16 4 2 8 4 ...
 $ premium: Factor w/ 2 levels "no","yes": 2 2 2 1 2 2 2 2 2 2
```

O arquivo de dados computadores será separado em subarquivo treinamento e teste, utilizando uma razão igual a 0,80. O subarquivo treinamento será composto por 80% das observações (5.007 observações) e o restante dos 20% (1.252 observações) irá compor o teste.

```
set.seed(123)
n <- nrow(computadores)
indices <- sample(n, n * 0.8)
treino <- computadores[indices, ]
teste <- computadores[-indices, ]
```

A variável resposta (*premium*) é se o fabricante do computador era uma empresa "premium" (IBM, COMPAQ), cuja resposta é "yes" ou "no", em função das variáveis *price* (preço do computador em U\$ dólar), *speed* (velocidade do relógio (MHZ)), *hd* (tamanho do disco rígido em MB) e *ram* (tamanho da ram em MB). Observem que as variáveis preditivas são numéricas, portanto, o teste em um nó determina se o valor é maior ou menor que um determinado valor, definido pelo próprio *software R*.

```
install.packages("rpart") # instala a biblioteca rpart
install.packages("rpart.plot") # instala a biblioteca rpart.plot
library(rpart)
library(rpart.plot)
set.seed(123)
modelo <- rpart(premium ~ price + speed +hd +ram,
                 method="class", data=treino)
```

Para avaliar o modelo basta utilizar o subarquivo treinamento e efetuar a previsão.

```
ajuste<-predict(modelo,treino,type="class")
```

Desejando-se obter a taxa de acerto na classificação basta fazer:

```
treino$premium<-as.factor(treino$premium)
tabela<-table(ajuste,treino$premium);tabela
```

ajuste	no	yes
no	364	65
yes	120	4458

A taxa de acerto é:

$$taxa\ de\ acerto = \frac{364 + 4458}{5.007} = 0,9631 \times 100 = 96,31\%$$

Para realizar a classificação das observações do subarquivo teste, basta usar a função *predict*, modelo e o arquivo teste.

```
predicao <- predict(modelo, teste, type="class")
```

O subarquivo teste contendo a classificação obtida com a aplicação da árvore de decisão poderá ser visualizado como apresentado a seguir (resultado). A última coluna é o resultado da aplicação da árvore de decisão.

```
predicao <- predict(modelo, teste, type="class")
resultado<-cbind(teste,predicao)
resultado[1:5,]
```

	price	speed	hd	ram	premium	predicao
8	1995	50	85	2	yes	yes
9	2225	50	210	8	yes	yes
14	2295	25	245	8	yes	yes
18	2325	33	210	4	yes	yes
31	1920	33	170	4	yes	yes

A taxa de acerto da previsão é obtida por meio de:

```
teste$premium<-as.factor(teste$premium)
tabela<-table(predicao, teste$premium); tabela
```

predicao	no	yes
no	85	17
yes	43	1107

$$taxa\ de\ acerto = \frac{85 + 1107}{1.252} = 0,9521 \times 100 = 95,21\%$$

Para classificar novas observações, deve-se criar o objeto contendo os valores das variáveis.

```
price<-c(2200,2800,2000)
speed<-c(50,33,25)
hd<-c(170,240,200)
ram<-c(8,4,2)
x=data.frame(price,speed,hd,ram)
```

```
x
  price speed  hd ram
1  2200    50 170   8
2  2800    33 240   4
3  2000    25 200   2
predict(modelo,x,type="class")
```

As novas observações receberão as seguintes classificações:

```
1  2  3
yes no yes
```

REFERÊNCIAS

CASTRO, L. N. de; FERRARI, D. G. **Introdução à mineração de dados:** conceitos básicos, algoritmos e aplicações. São Paulo: Saraiva, 2016.

CIABURRO, G.; VENKATESWARAN, B. **Neural networks with R:** Smart models using CNN, RNN, deep learning and artificial intelligence principles. Birmingham: Packt Publishing Ltd, 2017.

FINOCCHIO, M. A. F. **Noções de redes neurais artificiais.** Notas de aula. Julho de 2014. Disponível em: <<http://paginapessoal.utfpr.edu.br/mafinocchio/labsi-laboratorio-de-seguranca-e-iluminacao/redes-neurais-artificiais/NOCaO%20DE%20REDES%20NEURAIIS%20ARTIFICIAIS.pdf/view>> Acesso em: 14 abr. 2021.

IBGE. Instituto Brasileiro de Geografia e Estatística. **Pesquisa Industrial mensal.** Disponível em: <http://ipeadata.gov.br/beta3/#!/dados-serie?anomapa=&ascOrder=&base=macro&busca=&columnOrdering=&end=2020&fonte=&serid=PIMPFN12_QIITN12&skip=0&start=2002&tema=Produ%C3%A7%C3%A3o&territoriality=>>. Acesso em: 14 abr. 2021.

QUILLICI-GONZALEZ, J. A.; ZAMPIROLI, F. de A. **Sistemas inteligentes e mineração de dados.** Santo André: Triunfal Gráfica e Editora, 2014. Disponível em:

<https://books.google.com.br/books?id=X76VBgAAQBAJ&printsec=frontcover&hl=pt-BR&source=gbs_ge_summary_r&cad=0#v=onepage&q&f=false>. Acesso em: 14 abr. 2021.

ROCHA, D. **Introdução a Machine Learning:** Knn. 2019. Disponível em: <https://rpubs.com/davimat/knn_no_r>. Acesso em: 14 abr. 2021.

SILVA, I. N. da; SPATTI, D. H.; FLAUZINO, R. A. **Redes neurais artificiais:** para engenharia e ciências aplicadas. São Paulo: Artliber, 2010.

SILVA, L. A. da; PERES, S. M.; BOSCARIOLI, C. **Introdução à mineração de dados: com aplicações em R.** 1. ed. Rio de Janeiro: Elsevier, 2016.

SISTEMA R. **Biblioteca datasets.** Disponível em:

<<https://vincentarelbundock.github.io/Rdatasets/datasets.html>>. Acesso em: 14 abr. 2021.

SISTEMA R. **Biblioteca *Ecdat***. Disponível em:

<<https://vincentarelbundock.github.io/Rdatasets/datasets.html>>. Acesso em: 14 abr. 2021.

SISTEMA R. Disponível em: <<http://cran.r-project.org>>. Acesso em: 14 abr. 2021.